Build an AI Research Assistant Using LangChain

Objective

The goal of this project is to build a production-quality AI Research Assistant using **LangChain**. The assistant should be capable of understanding documents, answering questions, using tools, maintaining conversation context, and coordinating multiple reasoning steps through LangChain components.

The emphasis of this project is **learning LangChain**, not building a polished application.

Learning Objectives

By the end of this project, you should understand:

- Prompt Templates
- Chat Models
- Output Parsers
- LCEL (LangChain Expression Language)
- Runnable Sequences
- Runnable Parallel
- Chains
- Document Loaders
- Text Splitters
- Embeddings
- Vector Stores
- Retrievers
- Retrieval-Augmented Generation (RAG)
- Conversation Memory
- Custom Tools
- Tool Calling
- Agents
- Multi-Agent Workflows
- Structured Output
- Streaming Responses
- Callbacks
- LangGraph Integration
- Human-in-the-Loop Workflows

- Checkpointing
- Context Engineering

Problem Statement

Build an AI Research Assistant capable of answering questions from a collection of documents while intelligently deciding when to retrieve information, invoke tools, maintain memory, or delegate tasks to specialized agents.

The assistant should behave like an intelligent research assistant that can reason through complex queries using LangChain's ecosystem.

Project Phases

Phase 1 – Basic Chatbot

Concepts

- Chat Models
- Prompt Templates
- RunnableSequence
- Output Parsers

Requirements

- Accept user questions.
- Generate responses using an LLM.
- Use Prompt Templates instead of raw prompts.
- Parse responses into structured formats when required.

Deliverable:

A conversational chatbot implemented entirely using LangChain.

Phase 2 – LCEL Pipelines

Concepts

- RunnableSequence
- RunnableParallel
- RunnablePassthrough

Requirements

Create reusable pipelines using LCEL.

Examples:

- Question → Prompt → LLM → Parser
- Question → Parallel chains → Combined response

Deliverable:

At least three LCEL pipelines demonstrating different execution patterns.

Phase 3 – Document Loading

Concepts

- Document Loaders

Requirements

Load documents from multiple formats such as:

- PDF
- Markdown
- TXT
- HTML

The system should create a unified document collection.

Deliverable:

A document ingestion pipeline.

Phase 4 – Text Splitting

Concepts

- Recursive Text Splitters
- Token-based Splitters

Requirements

Split documents into optimal chunks.

Experiment with different chunk sizes and overlaps.

Deliverable:

Chunked document dataset ready for embedding.

Phase 5 – Embeddings and Vector Store

Concepts

- Embeddings
- Vector Store

Requirements

Generate embeddings for all chunks.

Store them in a vector database supported by LangChain.

Deliverable:

A searchable knowledge base.

Phase 6 – Retrieval-Augmented Generation (RAG)

Concepts

- Retriever
- Retrieval Chain

Requirements

The assistant should answer questions using retrieved context instead of relying solely on the language model.

Deliverable:

A complete RAG pipeline.

Phase 7 – Conversation Memory

Concepts

- Chat Message History
- Memory

Requirements

The assistant should remember previous interactions and use that context in future responses.

Example:

User: My favorite programming language is Python.

Later:

User: Generate an example.

Assistant should automatically use Python.

Deliverable:

Persistent conversational context during a session.

Phase 8 – Custom Tools

Concepts

- Tool Interface
- Tool Calling

Requirements

Create at least five custom LangChain tools.

Suggested examples:

- Calculator
- Current Date & Time
- Text Summarizer
- Keyword Extractor
- Document Search
- Word Counter

The assistant should choose the correct tool automatically.

Deliverable:

Five working tools integrated with an agent.

Phase 9 – Agent

Concepts

- Agent
- Tool Calling

Requirements

Build an agent capable of deciding:

- When to answer directly
- When to retrieve documents
- When to invoke tools

Deliverable:

A fully functioning LangChain agent.

Phase 10 – Structured Output

Concepts

- Output Parsers
- Pydantic Models

Requirements

Return structured responses for specific tasks.

Example:

```
{
  "summary": "...",
  "keywords": [],
  "confidence": 0.91
}
```

Deliverable:

At least three structured output formats.

Phase 11 – Streaming

Concepts

- Streaming

Requirements

Display generated tokens as they are produced.

Deliverable:

Streaming-enabled responses.

Phase 12 – Multi-Agent Workflow

Concepts

- Multiple Agents
- Agent Collaboration

Requirements

Create specialized agents.

Suggested agents:

- Research Agent
- Summarization Agent
- Fact Checking Agent
- Question Answering Agent
- Report Writing Agent

A coordinator agent should decide which agent(s) should handle each request.

Deliverable:

A collaborative multi-agent workflow.

Phase 13 – LangGraph Workflow

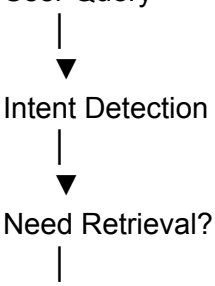
Concepts

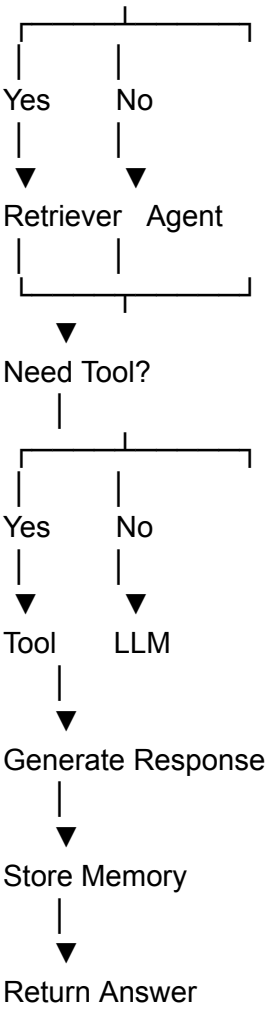
- LangGraph
- State Management
- Conditional Edges

Requirements

Implement the complete assistant as a graph.

Suggested flow:





Deliverable:

A complete LangGraph implementation.

Phase 14 – Human-in-the-Loop

Concepts

- Interrupts
- Checkpointing

Requirements

Before executing predefined sensitive actions, pause execution and request user confirmation.

Deliverable:

Human approval integrated into the LangGraph workflow.

Phase 15 – Context Engineering

Concepts

- Context Selection
- Prompt Construction

Requirements

Construct prompts by combining:

- User query
- Conversation history
- Retrieved documents
- Tool outputs

The assistant should include only the most relevant context.

Deliverable:

An optimized context assembly pipeline.

Expected Folder Structure

You can modify if you like.
project/

├── app.py
├── prompts/
├── chains/
├── agents/
├── tools/
├── memory/
├── retrieval/
├── loaders/
├── embeddings/

```
├── vectorstore/
├── parsers/
├── graph/
├── callbacks/
├── models/
├── utils/
├── data/
└── tests/
```

Bonus Tasks

- Add retry mechanisms using RunnableRetry.
- Compare multiple retrievers.
- Implement conversational RAG.
- Build reusable prompt libraries.
- Add callback handlers for logging.
- Benchmark different chunk sizes.
- Implement hybrid retrieval.
- Create custom output parsers.
- Add custom Runnable components.
- Optimize prompt and retrieval performance.

The primary objective is to develop a deep understanding of LangChain's architecture and programming model by building a complete, end-to-end AI application.